Twelve-minute recording script

FTEC5660 - Individual Project 2 - Reproducibility

SANG Yuchen | 1155262204

Target duration: approximately 12 minutes, including screen changes. The assignment limit is 15 minutes. The spoken text contains approximately 1,490 words. Read only the paragraphs marked Say; screen directions are not spoken. Aim for 135-145 words per minute, allowing about 60-90 seconds for screen changes, and rehearse once with a timer. The timestamps are pacing targets, not a guarantee of your speaking time. At a slower pace, shorten examples during rehearsal rather than rushing.

Prepare before recording

1. Open FTEC5660_Presentation.pdf in a PDF viewer, starting at page 1. This remains the main presentation.

2. Open FTEC5660_LiTS_DeepSeek_Report.pdf, with pages 1, 3 and 4 easy to reach.

3. In a browser, open the public submission repository, its commit history, assignment/experiment_plan.json, assignment/study.py, lits/lm/openai_chat.py, assignment/DEBUG_DIARY.md, and assignment/evidence/steps1_seed11/.

4. Open assignment/deliverables/formal_trials.csv and assignment/deliverables/format_audit.json. Use the browser's code view for JSON/CSV if necessary.

5. In a terminal at the repository root, prepare the offline command .\.venv\Scripts\python.exe -m assignment.study preflight. Run it once before recording to check the environment. This command makes no model calls. Keep the successful output available as a fallback.

6. Close .env, chats containing keys, and unrelated tabs. Use Zoom screen sharing and recording. Share the application windows or desktop needed for the demonstration. Keep this script outside the shared view if possible.

Screen plan

Time	Main slide	Open while speaking	Show / point to
00:00-00:45	1	Slides only	Name, student ID, research question
00:45-02:00	2	Report page 1; then return to slide 2	Table 6 reference; workflow and comparability caveat
02:00-03:15	3	assignment/experiment_plan.json	Source revision, dataset selection, model and endpoint
03:15-04:30	4	Plan study section; assignment/study.py	Caps 1/3/10; seeds; forced depth bound
04:30-05:40	5	formal_trials.csv; report page 3	Nine trials; primary mean and sample SD
05:40-06:40	6	Plot on slide 6	Token tradeoff; timing caveat
06:40-07:45	7	Report page 3 observed-depth table	Deepest node 4 at cap 10
07:45-09:00	8	lits/lm/openai_chat.py; assignment/DEBUG_DIARY.md	DeepSeek adapter; Windows failures and retained attempts
09:00-10:40	9	Evidence folder; format_audit.json	Manifest, request log, tree bundle; unit/currency examples
10:40-12:00	10	GitHub README and commits; terminal preflight	Reproduction steps; successful offline verification

00:00-00:45 - Slide 1: Introduction

Screen: Start Zoom recording with slide 1 visible. Point to the research question.

Say:

Hello, my name is SANG Yuchen, and my student ID is 1155262204. This is my reproducibility project for FTEC5660. I selected LiTS, a modular framework for language-model tree search. I ran the authors' mathematical reasoning workflow through the DeepSeek API, then investigated one question: how does maximum search depth affect accuracy and resource use? I compared three depth limits using the same thirty questions and three repeated runs for each setting. I will explain the setup, show the measured results, and demonstrate where the evidence and reproduction instructions are stored.

00:45-02:00 - Slide 2: What I attempted to reproduce

Screen: Show slide 2. Switch to report page 1 at “Objective and scope”; highlight the paragraph about Table 6. Return to slide 2.

Say:

LiTS qualifies as an agentic system because it searches over alternative reasoning trajectories across multiple steps. A policy proposes candidate reasoning actions, a transition component extends the trajectory, and a reward component evaluates those candidates. Monte Carlo Tree Search decides which branches to explore. The saved terminal answers then pass through the repository's voting and numeric-checking workflow.

My reference is Table six of the paper, where ReST with MCTS reports thirty-seven percent accuracy on one hundred numerical-answer MATH500 questions. My experiment uses DeepSeek, an API-based generative reward component, and a smaller subset. Therefore, my scores are not directly comparable with that number. I am reproducing the runnable workflow and its evaluation behavior under documented substitutions. I am not claiming that my higher score proves DeepSeek is better than the original model. The controlled comparison in this project is between depth settings within the same DeepSeek setup.

02:00-03:15 - Slide 3: Fixed environment, data and model

Screen: Show slide 3, then open assignment/experiment_plan.json. Point to upstream_commit, dataset, and model; do not read the long hashes aloud.

Say:

Here is the saved experiment plan. The upstream source is pinned to a specific Git commit, and the dataset is pinned to a specific Hugging Face revision. The source file contains three hundred and sixteen numeric-answer rows. I used the first thirty in their original order, rather than choosing questions based on their results. The dataset checksum is verified when loading.

The environment is an isolated Python three point twelve installation on Windows. I installed the dependencies needed for the API mathematics workflow. The recorded model name is deepseek-flash, served through the DeepSeek endpoint, with internal thinking disabled. The experiment logs actual request parameters and token usage because different roles use different decoding settings. Local seeds control local randomness, but they do not make a hosted model deterministic. Pinning the source and data improves reproducibility; it cannot freeze the provider's future model behavior.

03:15-04:30 - Slide 4: The controlled modification

Screen: Show slide 4, then the plan's study section. Open assignment/study.py and search for force_terminating_on_depth_limit=True.

Say:

The main experimental factor is maximum steps: one, three, or ten. Ten is the baseline cap in this study. Each setting uses the same thirty questions and three local seeds: eleven, twenty-two, and thirty-three. That gives nine complete runs and two hundred and seventy question-runs, although there are only thirty distinct questions.

The other search settings stay fixed: ten MCTS iterations, branching factor three, and rollout length two. A crucial implementation detail is shown here: forced termination at the depth limit is enabled for every group. Without that option, changing the configured maximum would not reliably enforce the intended bound.

A five-question pilot informed the grid before formal execution. The initial grid was five, ten, and fifteen, but the pilot often stopped at depth one. I changed it to one, three, and ten to include a binding short budget. The pilot overlaps with the formal subset, so this remains an exploratory study.

04:30-05:40 - Slide 5: Accuracy and trial-level results

Screen: Show slide 5. Briefly open formal_trials.csv to show all nine rows, then report page 3 for the aggregate table.

Say:

These results come from completed runs, not illustrative examples. The CSV contains one row for every depth and seed combination. Accuracy uses all thirty planned questions in each completed run. Missing final answers count as incorrect.

At depth one, mean accuracy is ninety percent, with zero sample standard deviation across the three runs. At depth three, the mean is also ninety percent, with a standard deviation of three point three percentage points. At depth ten, the mean is eighty-five point six percent, with a standard deviation of five point one percentage points.

These deviations describe variation between repetitions on the same small set. They are not confidence intervals for all of MATH500. The zero deviation at depth one means that the three scores matched; it does not establish that every generated response was identical. In this subset, increasing the cap did not produce a consistent accuracy improvement.

05:40-06:40 - Slide 6: Resource tradeoffs

Screen: Keep slide 6 visible. Point first to accuracy, then tokens, then time.

Say:

The plot makes the resource tradeoff clearer. Depth one uses about eight thousand four hundred and sixty-one tokens per question. Depth three uses about ten thousand nine hundred and eighty-four, and depth ten uses about eleven thousand three hundred and seventy-three. These totals include both search and final answer voting. Depth one uses roughly twenty-six percent fewer tokens than depth ten in these runs.

Average recorded time is about fourteen point six, sixteen point nine, and seventeen point five seconds per question. However, timing is descriptive. The initial batch ran nine processes, while one replacement run executed alone after a Windows file-write failure. Those load conditions differ, so I would not interpret the time differences as a controlled benchmark of latency. Tokens provide a clearer resource measure here; they are not a calculated monetary bill.

06:40-07:45 - Slide 7: What does one step mean?

Screen: Show slide 7. Switch to the observed-depth table on report page 3; point to cap 10 / deepest node 4. Return to slide 7.

Say:

Why can a one-step setting solve these problems? One external search action can contain a complete derivation with several mathematical operations. The step cap therefore does not directly limit arithmetic

operations or internal reasoning tokens.

The observed-depth table supports this interpretation. The deepest saved node in the ten-step group is only at depth four. A cap of ten permits that much depth; it does not require every problem to take ten steps. Many trajectories terminate earlier. Calls also exceed tree depth because the system samples alternatives, scores candidates, and performs voting.

There is another qualification: the cap changes when LiTS supplies its final-step instruction. The measured intervention is the repository's depth-budget behavior, including that instruction. The results suggest that additional search can add cost without guaranteeing a better selected answer. They do not show that deeper reasoning is universally harmful.

07:45-09:00 - Slide 8: What worked and what failed

Screen: Show slide 8. In lits/lm/openai_chat.py, search for deepseek and strict. Then show assignment/DEBUG_DIARY.md, especially UTF-8 and completion-marker entries.

Say:

Getting the workflow running required several documented compatibility repairs. First, I prepared a compatible Python environment and removed an unused torch import from the API path. Second, the model wrapper needed a DeepSeek-specific mapping for its thinking option. The stable endpoint also required omitting a beta-only strict-tool flag, while retaining the schema and local validation. These repairs are shared across all formal groups.

On Windows, an early smoke run completed its API requests but failed while saving a Unicode character. The launcher now uses UTF-eight. Later, an initial depth-ten, seed-eleven attempt failed during an atomic completion-marker replacement. I retained that incomplete attempt and reran the entire configuration once with unchanged experimental source. I did not select a replacement based on accuracy.

The nine selected formal runs have matching recorded source fingerprints. A later monitor repair avoids opening active completion files. The diary distinguishes observed failures from possible explanations; concurrent file access is a suspected contributor, not a proven cause.

09:00-10:40 - Slide 9: Evidence, scoring and limitations

Screen: Show slide 9, then assignment/evidence/steps1_seed11/. Open run_manifest.json, api_calls.jsonl, and the start of full_trees.jsonl. Show assignment/deliverables/format_audit.json; point to one units/currency example.

Say:

The evidence is included so another reader can inspect more than the final table. This folder contains the configuration, run manifest, per-request token ledger, evaluation output, and bundles of full search trees and terminal nodes. The manifest records the run status and settings. The request log records actual model calls. The evidence index identifies the selected runs and excluded attempts. A restoration script rebuilds the raw run directories without making API calls.

One important limitation appeared in the scoring audit. The original numeric checker can penalize answers with units or currency. For example, seventy-two degrees may be rejected against the reference seventy-two, and twelve centimetres may be rejected against twelve. Across the formal runs, the audit identified sixteen such format-only false negatives in three known tasks. I kept the primary scores unchanged. This is a diagnostic finding, not a new scoring rule introduced to improve the results.

The study also has only thirty distinct questions, uses an ordered subset, and includes pilot overlap. The remote model alias may evolve, and reward scoring and answer grouping are themselves model-based. These constraints limit generalization. Small score differences may reflect formatting or selection behavior as well as reasoning. A stronger follow-up would use a larger held-out sample and a separately validated normalization

rule fixed before execution.

10:40-12:00 - Slide 10: Reproduction demonstration and conclusion

Screen: Show slide 10, then the public repository README and commit history. Point to install/run instructions and the report link. Switch to the prepared terminal; run .\.venv\Scripts\python.exe -m assignment.study preflight. Return to slide 10 for the final sentences. If preflight unexpectedly fails, show the saved successful output and describe it as a prior check.

Say:

The public repository provides the report, slides, experimental evidence, dependency lock, and installation and run instructions. The changes are separated into commits so the adapter, experiment setup, fixes, and results can be reviewed. API keys stay in the ignored local environment file.

Here I am running the offline preflight check. It verifies the prepared configuration and data without calling the model. Repeating the paid experiment requires a personal DeepSeek key and the documented formal command for each depth and seed. Inspecting the saved evidence and rebuilding the report do not require a key.

My conclusion is that this API-backed LiTS workflow runs under the documented conditions. On this subset, the default cap of ten scores eighty-five point six percent on average, while cap one scores ninety percent with lower token use. My recommendation is to measure actual depth and resource use before increasing a search budget, and to audit the evaluator before attributing small differences to reasoning. Thank you.

After recording

Stop recording, let Zoom finish processing, and watch the video to check audio, screen readability, and duration. Upload or share the recording through an accessible link. Replace the pending entry in VIDEO_LINK.md with the actual link and duration, then commit that update. Verify the teacher can access the link without your personal login. Submit the report PDF, public GitHub URL and recording link through the course's required submission channel before the stated deadline. A script or slide deck does not replace the recorded presentation.